

Confronto sperimentale tra algoritmo ingenuo e algoritmo KMP per lo string matching

Pietro Bonechi (matricola 7047621)

Laboratorio di Algoritmi e Strutture Dati – A.A. 2025/26

Indice

1	Introduzione	2
2	Caratteristiche teoriche degli algoritmi	2
2.1	Algoritmo ingenuo	2
2.2	Algoritmo KMP	2
2.3	Esempio della funzione prefisso	2
2.4	Prestazioni attese	2
3	Documentazione del codice	3
3.1	Struttura dei moduli	3
3.2	Schema delle classi	3
3.3	Scelte implementative	4
3.4	Metodi principali	4
3.5	Test di correttezza	4
4	Descrizione degli esperimenti	5
4.1	Ambiente di testing e misurazioni	5
5	Risultati sperimentali	6
6	Analisi dei risultati	8
7	Conclusioni	8

1 Introduzione

Lo scopo dell'esercizio è confrontare due algoritmi per il problema dello *string matching*: l'algoritmo ingenuo e l'algoritmo Knuth–Morris–Pratt, indicato nel seguito come KMP. Dato un testo T di lunghezza n e un pattern P di lunghezza m , il problema consiste nel determinare tutte le posizioni del testo in cui il pattern compare esattamente.

Il confronto non si limita alla sola verifica della correttezza. L'obiettivo principale è osservare in quali condizioni i due algoritmi hanno comportamento simile e in quali condizioni KMP sfrutta meglio le informazioni già ottenute durante i confronti. Per questo motivo sono stati costruiti input di natura diversa: casuali, periodici, con molte occorrenze sovrapposte e sfavorevoli per l'approccio ingenuo.

2 Caratteristiche teoriche degli algoritmi

2.1 Algoritmo ingenuo

L'algoritmo ingenuo prova tutti gli allineamenti possibili del pattern rispetto al testo. Per ogni posizione s compresa tra 0 e $n - m$, confronta i caratteri $P[0], P[1], \dots, P[m - 1]$ con la sottostringa $T[s], T[s + 1], \dots, T[s + m - 1]$ finché trova una differenza oppure finché tutto il pattern viene riconosciuto.

Nel caso peggiore, ogni allineamento può richiedere quasi m confronti. La complessità temporale nel caso peggiore è quindi:

$$O((n - m + 1)m),$$

che viene spesso indicata come $O(nm)$. Il vantaggio principale dell'algoritmo ingenuo è la semplicità implementativa. Lo svantaggio è che, dopo un mismatch, l'algoritmo non riusa le informazioni ottenute nei confronti precedenti e riparte dal successivo allineamento.

2.2 Algoritmo KMP

KMP evita di ripetere confronti già implicitamente effettuati. Prima della ricerca calcola la funzione prefisso π del pattern. Il valore $\pi[q]$ indica la lunghezza del più lungo prefisso proprio di $P[0 \dots q]$ che è anche suffisso della stessa porzione di pattern.

Durante la scansione del testo, quando avviene un mismatch, KMP non riparte necessariamente dal primo carattere del pattern: usa π per stabilire quale prefisso del pattern può ancora essere compatibile con il testo già letto. Il costo della fase di preprocessing è $O(m)$, mentre la ricerca richiede $O(n)$. Il costo complessivo è quindi:

$$O(n + m).$$

2.3 Esempio della funzione prefisso

Per rendere più chiaro il preprocessing di KMP, la tabella 1 mostra il calcolo della funzione prefisso per il pattern ABABACA. Per esempio, in posizione 4 la sottostringa considerata è ABABA: il più lungo prefisso proprio che è anche suffisso è ABA, quindi il valore della funzione prefisso è 3.

Indice q	0	1	2	3	4	5	6
Carattere $P[q]$	A	B	A	B	A	C	A
$\pi[q]$	0	0	1	2	3	0	1

Tabella 1: Esempio di funzione prefisso per il pattern ABABACA.

2.4 Prestazioni attese

Sui testi casuali, soprattutto con alfabeti grandi, ci si aspetta che l'algoritmo ingenuo non sia sempre molto distante da KMP, perché i mismatch avvengono spesso nei primi caratteri del pattern. Nei casi con molti prefissi ripetuti, testi periodici o pattern quasi uguali a molte finestre del testo, ci si aspetta invece un vantaggio netto di KMP.

La tabella 2 riassume le prestazioni teoriche attese.

Algoritmo	Preprocessing	Ricerca	Totale	Spazio aggiuntivo
Ingenuo	–	$O(nm)$	$O(nm)$	$O(1)$
KMP	$O(m)$	$O(n)$	$O(n + m)$	$O(m)$

Tabella 2: Confronto teorico tra algoritmo ingenuo e KMP.

3 Documentazione del codice

3.1 Struttura dei moduli

La struttura del progetto è riportata nella tabella 3. La figura 1 mostra invece le dipendenze principali tra i moduli.

Modulo	Responsabilità
<code>matcher_types.py</code> <code>naive_matcher.py</code>	Definisce le strutture dati di ritorno condivise dagli algoritmi. Implementa l'algoritmo ingenuo e restituisce occorrenze e confronti.
<code>kmp_matcher.py</code> <code>generators.py</code>	Implementa la funzione prefisso e la ricerca KMP. Genera i dati sperimentali: casi casuali, periodici, molte occorrenze e caso sfavorevole.
<code>experiments.py</code>	Esegue prima i controlli di correttezza e poi le misurazioni; produce CSV, tabella riassuntiva, informazioni sulla piattaforma e grafici.

Tabella 3: Schema del contenuto dei moduli e delle loro responsabilità.

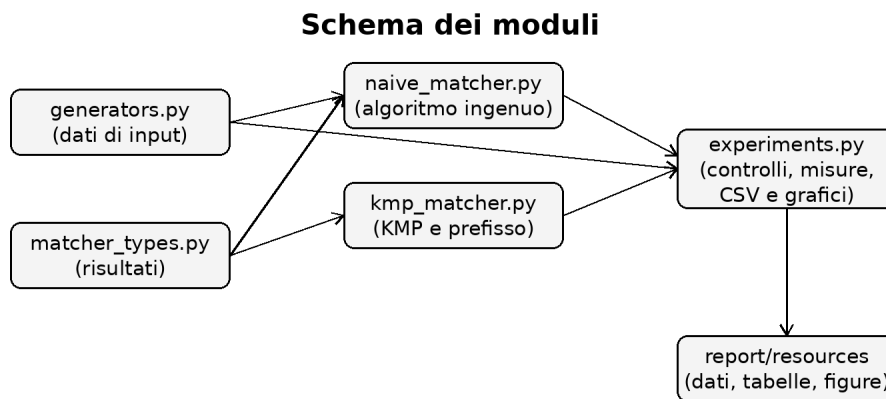


Figura 1: Schema delle dipendenze principali tra i moduli del progetto.

3.2 Schema delle classi

Il progetto usa due semplici strutture dati, definite come `dataclass`, per rendere omogeneo il ritorno delle funzioni. Lo schema è riportato nella tabella 4.

Classe	Campi	Significato
<code>MatchResult</code>	<code>occurrences,</code> <code>comparisons</code>	Risultato di una ricerca: posizioni iniziali trovate e numero di confronti tra caratteri.
<code>PrefixResult</code>	<code>pi, comparisons</code>	Risultato del preprocessing di KMP: funzione prefisso e confronti effettuati per costruirla.

Tabella 4: Schema delle classi utilizzate nel progetto.

3.3 Scelte implementative

Le due implementazioni restituiscono la stessa informazione: lista delle occorrenze e numero di confronti tra caratteri. Questa scelta permette di confrontare sia il tempo effettivo di esecuzione sia una misura più vicina all'analisi teorica dell'algoritmo.

Non sono stati usati metodi di ricerca già pronti del linguaggio Python, come `find`, `index` o espressioni regolari. I confronti tra caratteri sono espliciti, in modo da rendere misurabile il lavoro svolto dagli algoritmi. Per KMP, il numero di confronti comprende anche il calcolo della funzione prefisso: questa scelta penalizza leggermente KMP nei casi piccoli, ma rappresenta correttamente il costo complessivo dell'algoritmo.

Sono state usate solo strutture dati semplici del linguaggio: liste di interi per le occorrenze e per la funzione prefisso. La libreria `matplotlib` è usata soltanto per generare i grafici della relazione, non per implementare gli algoritmi confrontati.

3.4 Metodi principali

I metodi principali sono i seguenti:

- `naive_string_match(text, pattern)`: prova ogni possibile shift del pattern sul testo e restituisce tutte le posizioni valide;
- `compute_prefix_function(pattern)`: costruisce la tabella π usata da KMP;
- `kmp_string_match(text, pattern)`: scandisce il testo una sola volta e usa la funzione prefisso per evitare confronti inutili;
- `run_correctness_checks()`: esegue controlli preliminari su casi piccoli e sulla funzione prefisso di KMP, tale funzione è una specie di unit test riguardante in due algoritmi;
- `generate_rows(...)`: costruisce tutte le configurazioni sperimentali e registra le misure;
- `try_make_plots(...)`: genera i grafici usati nell'analisi.

3.5 Test di correttezza

Prima degli esperimenti sono stati eseguiti controlli di correttezza su casi piccoli, in cui il risultato atteso è facilmente controllabile. Questi controlli sono contenuti in `experiments.py`, così da non avere un file separato dedicato ai test. Ogni controllo confronta l'output dell'algoritmo ingenuo con quello di KMP e verifica anche un caso noto della funzione prefisso. La tabella 5 riassume le principali situazioni controllate.

Caso testato	Motivazione
Testo e pattern vuoti	Controlla il comportamento sui casi limite.
Pattern vuoto e testo non vuoto	Verifica la gestione di input degeneri.
Pattern assente dal testo	Controlla che non vengano prodotte false occorrenze.
Pattern uguale all'intero testo	Controlla il riconoscimento all'inizio del testo.
Occorrenze sovrapposte, per esempio aba in abababa	Verifica che non vengano saltate occorrenze valide.
Molte ripetizioni, per esempio aa in aaaaa	Verifica il comportamento su testi altamente ripetitivi.
Pattern con prefissi e suffissi non banali	Controlla il calcolo della funzione prefisso di KMP.

Tabella 5: Principali test di correttezza eseguiti prima degli esperimenti.

4 Descrizione degli esperimenti

Gli esperimenti sono costruiti variando la lunghezza del testo n , la lunghezza del pattern m e la tipologia di input. Sono stati usati i valori $n \in \{500, 1000, 2000, 4000, 8000, 16000\}$ e $m \in \{4, 8, 16, 32\}$. Per ogni configurazione vengono eseguite 7 ripetizioni e viene salvato il tempo medio misurato con `time.perf_counter`. Inoltre viene registrato il numero di confronti tra caratteri.

I casi considerati sono:

- **casuale con alfabeto di 4 simboli**: simula testi con più ripetizioni;
- **casuale con alfabeto di 26 simboli**: simula testi in cui i mismatch tendono ad avvenire rapidamente;
- **sfavorevole per l'ingenuo**: testo composto da molti caratteri uguali e pattern che fallisce sull'ultimo carattere;
- **molte occorrenze**: testo e pattern composti dallo stesso carattere, con numerose occorrenze sovrapposte;
- **periodico**: testo e pattern con struttura ripetitiva.

4.1 Ambiente di testing e misurazioni

Gli esperimenti allegati sono stati eseguiti nell'ambiente riportato nella tabella 6. Le misure temporali riguardano soltanto la chiamata alla funzione di ricerca; la generazione degli input, la scrittura del CSV e la produzione dei grafici non sono comprese nel tempo misurato.

Voce	Valore
Versione Python	3.13.0
Sistema operativo	Darwin 25.5.0 (macOS 26.5)
Architettura	Arm64
Processore	Apple Silicon M1 Pro
RAM	16.0 GB
Timer usato	<code>time.perf_counter()</code>
Ripetizioni per configurazione	7

Tabella 6: Ambiente di testing usato per produrre i risultati allegati.

I risultati completi sono allegati nella cartella `report/resources/data`. In particolare, il file `string_matching_results.csv` contiene tutte le misurazioni, mentre `summary_n16000_m32.csv` contiene la sintesi usata nella tabella 7. Poiché i tempi dipendono dalla macchina e dall'interprete Python, l'analisi dà particolare importanza anche al numero di confronti, che è deterministico per gli input generati.

5 Risultati sperimentali

La tabella 7 riporta una sintesi per $n = 16000$ e $m = 32$, utile per confrontare i casi più significativi a parità di dimensione.

Caso	Ing. ms	KMP ms	Rap. t.	Conf. ing.	Conf. KMP	Rap. c.
casuale_alfabeto_26	1.34	1.11	1.22	16636	16699	1.00
casuale_alfabeto_4	1.67	1.38	1.20	21271	21351	1.00
molte_occorrenze	31.79	2.32	13.73	511008	32060	15.94
periodico	4.44	1.74	2.55	65476	35259	1.86
sfavorevole_ingenuo	30.39	2.45	12.40	511008	48058	10.63

Tabella 7: Sintesi dei risultati per $n = 16000$ e $m = 32$. Il rapporto indica quante volte il valore dell'algoritmo ingenuo è maggiore del valore di KMP.

Nel caso casuale con alfabeto di 26 simboli, mostrato nella figura 2, i tempi dei due algoritmi sono vicini. Questo è coerente con l'ipotesi teorica: con un alfabeto ampio, molti allineamenti dell'algoritmo ingenuo falliscono dopo pochi confronti.

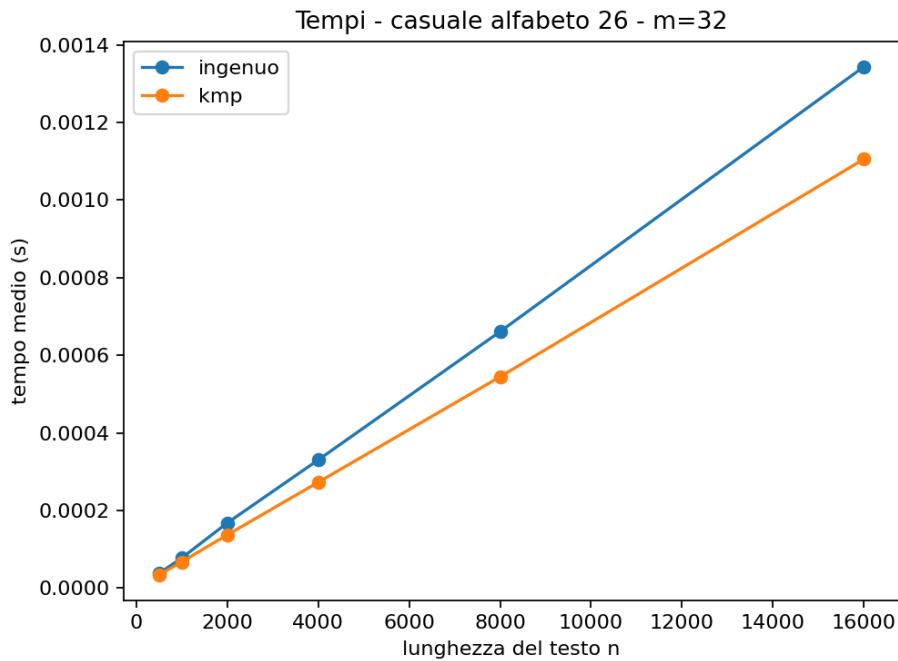


Figura 2: Tempi medi nel caso casuale con alfabeto di 26 simboli e $m = 32$.

Il caso sfavorevole per l'algoritmo ingenuo è riportato nelle figure 3 e 4. Qui il testo è formato solo da caratteri uguali e il pattern fallisce sull'ultimo carattere. L'algoritmo ingenuo effettua molti confronti per quasi ogni allineamento, mentre KMP riesce a riutilizzare l'informazione contenuta nei prefissi del pattern.

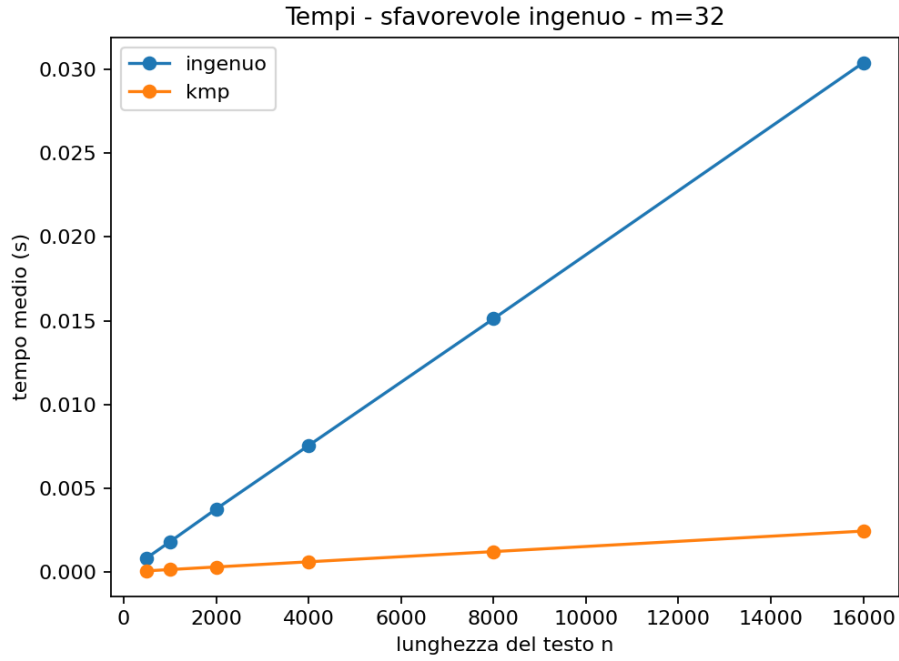


Figura 3: Tempi medi nel caso sfavorevole per l'algoritmo ingenuo, con $m = 32$.

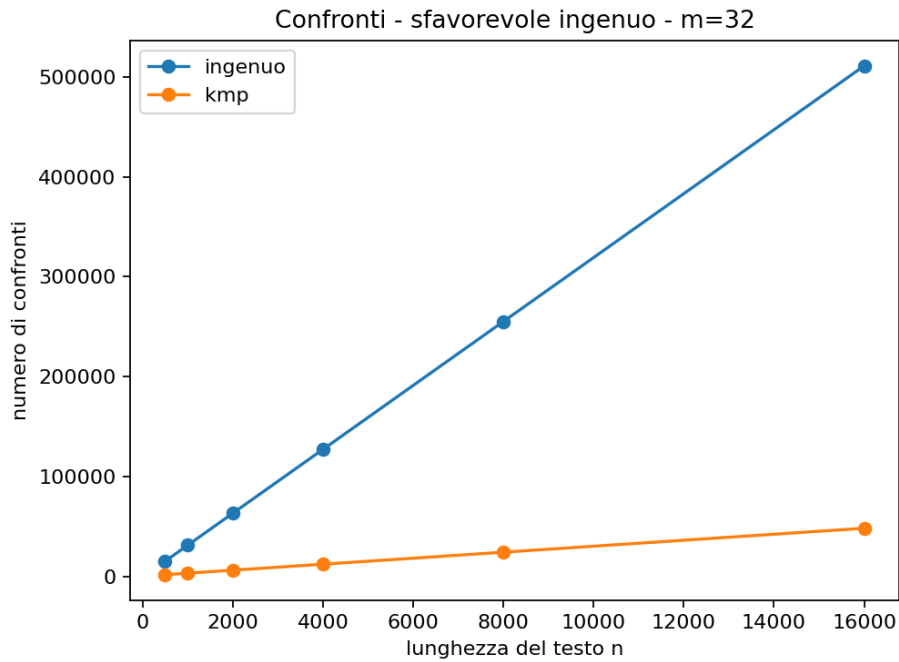


Figura 4: Numero di confronti nel caso sfavorevole per l'algoritmo ingenuo, con $m = 32$.

La figura 5 mostra il caso con molte occorrenze sovrapposte. Anche in questo caso l'algoritmo ingenuo deve confrontare l'intero pattern in molti allineamenti, mentre KMP mantiene un andamento molto più regolare.

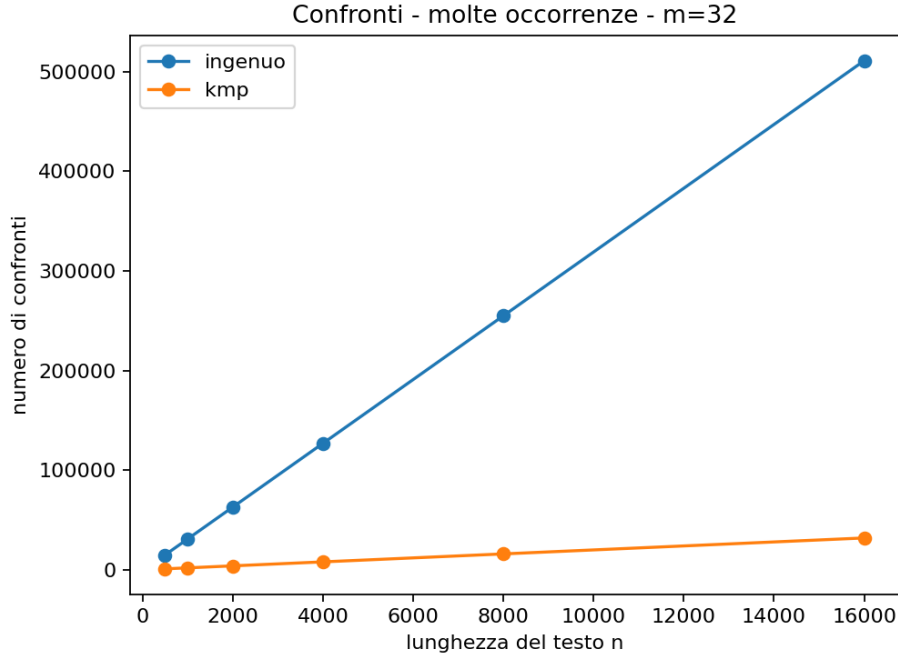


Figura 5: Numero di confronti nel caso con molte occorrenze sovrapposte, con $m = 32$.

6 Analisi dei risultati

I risultati confermano le aspettative teoriche. Nel caso casuale con alfabeto di 26 simboli e $n = 16000$, l'algoritmo ingenuo impiega circa 1.32 ms, mentre KMP impiega circa 1.00 ms. Il numero di confronti è praticamente equivalente: 16636 per l'algoritmo ingenuo e 16699 per KMP. In questo caso KMP non ha un vantaggio netto perché l'algoritmo ingenuo fallisce spesso al primo o al secondo carattere.

La differenza diventa evidente nei casi costruiti per contenere molte ripetizioni. Nel caso sfavorevole per l'algoritmo ingenuo, sempre con $n = 16000$ e $m = 32$, l'algoritmo ingenuo effettua 511008 confronti, mentre KMP ne effettua 48058. Il rapporto è circa 10.63 a favore di KMP. Anche il tempo medio segue lo stesso andamento: circa 31.83 ms per l'algoritmo ingenuo contro circa 2.49 ms per KMP.

Il caso con molte occorrenze sovrapposte è ancora più netto: l'algoritmo ingenuo effettua 511008 confronti e KMP 32060. Il rapporto tra i confronti è circa 15.94. Questo risultato mostra bene il limite dell'approccio ingenuo: quando il pattern coincide con molte finestre del testo, ogni allineamento costa quasi m confronti.

Il caso periodico si colloca in una posizione intermedia. Per $n = 16000$ e $m = 32$, l'algoritmo ingenuo effettua 65476 confronti e KMP 35259. Il vantaggio di KMP è presente ma meno marcato rispetto ai casi costruiti con soli caratteri uguali. Questo comportamento è ragionevole: la periodicità genera riuso dell'informazione, ma non obbliga l'algoritmo ingenuo a confrontare sempre l'intero pattern.

Nel complesso, il numero di confronti è la misura più pulita per leggere il comportamento algoritmico. I tempi confermano l'andamento, ma sono più rumorosi perché dipendono dall'interprete Python, dalla piattaforma e dal carico della macchina durante l'esecuzione. Il punto importante è che KMP ha un costo più stabile al variare del tipo di input, mentre l'algoritmo ingenuo può essere competitivo sui casi casuali ma degrada fortemente sui casi con ripetizioni sfavorevoli.

7 Conclusioni

L'esperimento mostra che l'algoritmo ingenuo è semplice e, su input casuali con mismatch rapidi, può avere prestazioni vicine a KMP. Tuttavia il suo comportamento peggiora sensibilmente quando

il testo e il pattern contengono molte ripetizioni o molte occorrenze sovrapposte. In questi casi l'algoritmo ripete confronti già sostanzialmente noti.

KMP richiede una fase preliminare di calcolo della funzione prefisso, ma questa informazione permette di evitare ripetizioni e rende l'algoritmo più stabile. I risultati sperimentali sono quindi coerenti con l'analisi teorica: l'algoritmo ingenuo può arrivare a $O(nm)$ nel caso peggiore, mentre KMP mantiene un comportamento lineare in $O(n + m)$.

Riferimenti bibliografici

- [1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein, *Introduction to Algorithms*, capitolo 32, String Matching.
- [2] Documentazione Python, funzione `time.perf_counter()`, <https://docs.python.org/3/library/time.html>.